LEETCODE PROBLEM 146: LRU CACHE SPECIFICATION

Antigravity Document Parsing Engine — Technical Standard v1.4

1. System Overview & Problem Statement

Design a data structure that follows the constraints of a Least Recently Used (LRU) cache.
The LRUCache class must be initialized with a positive capacity.

- get(key): Return the value of the key if the key exists, otherwise return -1.
- put(key, value): Update key value if present, or insert key-value pair into cache.
- Eviction: When capacity is reached, invalidate the least recently used key.

2. Algorithmic Constraints & Complexity Guarantees

- Time Complexity: O(1) average time complexity for both get and put operations.
- Space Complexity: O(capacity) bound using a doubly linked list + hash map.
- Key Range: 0 <= key <= 10^4, Value Range: 0 <= value <= 10^5.
- Concurrency: Thread-safe read/write locking with reentrant mutex.

3. Implementation Benchmark Matrix

Operation	Data Structure	Time Complexity	Memory Bound
Lookup (get)	Hash Map Index	O(1) Strict	48 bytes/node
Insert (put)	Doubly Linked List	O(1) Strict	64 bytes/node
Evict Tail	Pointer Re-link	O(1) Strict	0 alloc overhead